

Amptek Mini-X Controller — Communication Protocol

Reverse-engineered from `MiniXDlg.cpp` / `MiniXDlg.h` (Amptek MFC reference application) and validated empirically against hardware.

Document date	13 August 2026
Reference source	<code>MiniXDlg.cpp</code> , <code>MiniXDlg.h</code> , board revision C0
Validated on	Mini-X Controller, serial 01300036, 10 W rating (verified from hardware documentation, not from software), 50 kV board, macOS, pyftdi
Status	All signal paths confirmed working except as noted in §10. Power-limit behavior documented for both 4 W and 10 W variants; only 10 W was exercised — see §6.1.1 and §12.

1. Overview

The Mini-X controller is a USB device built around an **FTDI FT2232C/D** in **MPSSE** (Multi-Protocol Synchronous Serial Engine) mode. There is no packet-oriented command language: the host synthesizes bit-level SPI-like waveforms by writing MPSSE opcode streams, and the FTDI chip drives the peripherals directly.

Four peripherals share the FT2232’s channel-A pins:

Peripheral	Part	Function	Chip select	Polarity
Dual DAC	AD5623R	HV and current setpoints	DACS (ADBUS4)	Active low
ADC	MAX186-family	HV and current monitors	ADCS (ADBUS3)	Active low
Temp sensor	DS1722	Board temperature	TSCS (ACBUS3)	Active high
GPIO	—	HV enables, interlock, status	direct pin access	see §7

All four multiplex a single clock and a single bidirectional data line. Chip selects are managed explicitly by the host; there is no hardware arbitration. **All transactions must be serialized** — see §11.

2. USB identification

Field	Value
Vendor ID	0x0403 (FTDI)
Product ID	0xD058 — custom Amptek PID
Description	MiniX Controller
Serial number	8 ASCII digits, e.g. 01300036
Interface	Channel A (/1 in pyftdi URL syntax)
device_version	0x0500 (FT2232C/D)

The custom PID is significant: generic FTDI tools that filter on known PIDs will **not** enumerate this device until the PID is registered.

```
Ftdi.add_custom_product(0x0403, 0xD058, 'minix')
```

The reference application locates the device by *description* rather than VID/PID, which is why the custom PID never affected it. When enumerating via FTD2XX, the description carries a trailing " A" / " B" channel suffix that the reference code strips with `ChCount -= 2`; pyftdi reports the base string without the suffix.

2.1 Serial number selects the calibration table

```
serial > 9999          -> isNSI = true   (newer hardware, longer settling delays)
serial >= 1118880      -> is50kv = true  (50 kV table)
otherwise              -> 40 kV table
```

Serial 01300036 → 1300036 → **NSI, 50 kV**.

The serial number does not encode the power rating. It gates exactly two booleans, neither of which touches power:

- `isNSI` → settling delays
- `is50kv` → `HighVoltageMax`, `HighVoltageConversionFactor`, and which iso-curve bitmap is displayed

The two setup routines (`ReadMiniXSetup()` / `ReadMiniXSetup50kv()`) are otherwise identical, and both contain `WattageMax = 4.00` as a literal. There is no serial-number threshold, no strapping pin, and no register anywhere in the protocol that reports the rating. Voltage range and power rating are **independent** axes of configuration, and only the former is discoverable at runtime.

The rating must therefore be supplied from outside the protocol — hardware label, shipping documentation, or the vendor. Do not infer it. See §6.1.1.

Unexplored: `MiniXDlg.h` declares `EE_Read` / `EE_Program` wrappers around the FT2232 EEPROM, which the reference application never uses for configuration. Whether the vendor stores a rating in the EEPROM user area is unknown; a read-only dump would settle it. Treat as curiosity, not as a source of truth.

3. Pin map (revision C0)

ADBUS (low byte)

Bit	Mask	Name	Dir	Function
7	0x80	MONX	In	MON MINIX RDY status
6	0x40	HVEN2	Out	CTRL_HV_EN_B
5	0x20	HVEN	Out	CTRL_HV_EN_A
4	0x10	DACS	Out	DAC chip select (active low)
3	0x08	ADCS	Out	ADC chip select (active low)
2	0x04	DataIN	In	MISO
1	0x02	DataOUT	Out	MOSI (DATASTATE)
0	0x01	CLK	Out	SCLK (CLKSTATE)

Direction byte: `OUTPUTMODE = INPUTMODE = 0x7B` (0111 1011).

Note both constants are identical in revision C0 — the direction byte never actually changes. The `INPUTMODE` writes in the ADC read path are effectively no-ops that only rewrite the state byte. Earlier board revisions (A0/A1/B0) used distinct values and did flip D1's direction; those revisions are documented in the source but are not covered here.

ACBUS (high byte)

Bit	Mask	Name	Dir	Function
3	0x08	TSCS	Out	Temp sensor chip enable (active high)
2	0x04	?	In	Unidentified , observed set
0	0x01	!RESET	In	Interlock: 1 = closed, 0 = open

Direction byte: `OUTPUTMODE_H = 0x08` — only ACBUS3 is an output.

4. MPSSE opcodes used

Opcode	Operands	Meaning
0x80	state, dir	Set ADBUS data + direction
0x82	state, dir	Set ACBUS data + direction
0x81	—	Read ADBUS (returns 1 byte)
0x83	—	Read ACBUS (returns 1 byte)
0x86	0x03 0x00	Set clock divisor $\rightarrow \sim 1.5$ MHz
0x10	lenL, lenH, data...	Clock bytes out, MSB first
0x13	len, data	Clock bits out, MSB first
0x20	lenL, lenH	Clock bytes in, MSB first

Length fields are $n-1$: 0x02 0x00 = 3 bytes, 0x01 0x00 = 2 bytes, 0x03 (bits) = 4 bits.

Naming caveat. The reference source defines CLK_FN_NEG 0x10 and CLK_FN_POS 0x11, then selects CLK_FN = CLK_FN_NEG. These names are **swapped** relative to FTDI AN_108, and inline comments compound the confusion. The numeric opcodes are authoritative and have been validated on hardware (§10). Ignore the edge annotations in the original source.

Invalid alternatives in the original. ReadTemperature() contains commented-out alternatives 0x22, 0x24, 0x26 alongside 0x20. Of these, 0x22 and 0x26 are *bit-mode* reads taking a **single** length byte; as written with two length bytes they would leave a stray 0x00 to be parsed as a command, which MPSSE rejects. Only 0x20 and 0x24 are valid byte-mode reads.

5. Initialization

1. FT_SetLatencyTimer(4) # 4 ms
2. FT_SetTimeouts(40, 40) # ms read/write
3. purge buffers
4. FT_SetBitMode(0x00, 0x02) # enable MPSSE
5. 0x82, high_state, 0x08 # ACBUS: TSCS deasserted (low)
6. 0x80, 0xFB & ~0x20 & ~0x40, 0x7B # ADBUS: HV enables CLEARED
7. 0x86, 0x03, 0x00 # clock divisor

Step 6 is the safety-critical one: the initial ADBUS state is 0xFB (all high) with **both HV enable bits explicitly cleared**, so bringing up the link cannot energize the tube.

To tear down: FT_SetBitMode(0x00, 0x00).

5.1 Health check

Sending an invalid opcode makes MPSSE echo 0xFA followed by the offending byte. Confirmed working:

```
send 0xAB -> receive 0xFA 0xAB
```

Useful for verifying synchronization before trusting any read.

5.2 Full startup sequence

```
enumerate -> open -> MPSSE init -> SetTempSensor -> SetClkDivisor  
          -> HV DAC = 0 -> (200 ms) -> current DAC = 0 -> (2 s)
```

SetTempSensor() sets its own clock divisor inline, making the subsequent SetClkDivisor() redundant. Harmless.

6. Analog channels

6.1 Scaling constants

Three parameters vary between units. Two are selected by serial number; the third is not.

Voltage range (from is50kv)

Constant	40 kV board	50 kV board
HighVoltageConversionFactor	10.0 kV/V	12.5 kV/V
HighVoltageMax	40.0	50.0
HighVoltageMin	9.999999	9.999999

Rating-independent (identical on every unit)

Constant	Value
VRef	4.096 V
DAC_ADC_Scale	4096 (12-bit)
CurrentConversionFactor	50.0 μ A/V
CurrentMin	4.999999 (0.999999 under _RELEASE_1uA)
CurrentMax	200.0
SafetyMargin	0.050 W

Note that **nothing in the DAC or ADC path varies with the power rating**. Full scale is 4.0 V $\rightarrow$ 4000 counts on both channels regardless: 50 kV and 200 μ A, whose product is exactly 10 W. Resolution is 0.0125 kV and 0.05

µA per count. The analog path is sized for the higher rating on every unit; only the software clamp differs.

The minima are stored as 9.999999 and 4.999999, not 10.0 / 5.0. This is deliberate: values are clamped against the minimum and then formatted "%.0f", so the epsilon lets a user entry of exactly 5.0 pass the < comparison while still displaying as "5". Preserve verbatim.

6.1.1 Power rating The vendor ships both 4 W and 10 W variants. **The rating is a compile-time literal in the reference application and is not discoverable at runtime (§2.1).**

Constant	4 W variant	10 W variant
WattageMax	4.00 W	10.00 W
SafetyMargin	0.050 W	0.050 W as shipped — see below
SafeWattageMW	3950 mW	9950 mW
mwCaution (§6.4)	3900 mW	9900 mW
mwDanger (§6.4)	4000 mW	10000 mW

The reference source contains only the 4 W values. The 10 W column follows from substituting `WattageMax = 10.00` into the same expressions; **it is derived, not transcribed.**

Consequence for the safety margin. `SafetyMargin` is absolute, not proportional. At 4 W, 50 mW is 1.25 % of full rating; at 10 W it is 0.5 %. Since §10.3 measured +4 % readback error on the current channel at low setpoints, a fixed 50 mW margin is proportionally *tighter* on the higher-rated unit while the measurement uncertainty is unchanged. Consider scaling the margin with the rating (125 mW at 10 W preserves the 1.25 % relationship) and document whichever choice you make, since the two units will then disagree on where the caution band begins.

Asymmetry of error. These two mistakes are not equivalent:

- Configuring **10 W on a 4 W unit** permits the operator to command 2.5× the tube’s rating. The software clamp is the only thing preventing it — the analog path will oblige, and the range checker (§6.4) will report the tube as *in range* the whole time, because it validates monitor against setpoint, not setpoint against rating.
- Configuring **4 W on a 10 W unit** costs flux and nothing else.

When the rating is not confirmed, 4.00 is the correct value to assume.

Implementation. Do not carry a default. Require the rating explicitly at construction and fail loudly if it is absent:

```
class MiniX:
    def __init__(self, ftdi, *, watt_max: float, hv_max: float, ...):
```

```

if watt_max not in (4.0, 10.0):
    raise ValueError(
        f"watt_max={watt_max}: must be 4.0 or 10.0, from the unit's "
        f"hardware documentation. There is no autodetect (§2.1)."
    )
self.watt_max = watt_max
self.safe_mw = (watt_max - self.safety_margin) * 1000.0

```

Store the rating alongside the tube serial number in whatever configuration file the GUI reads, and **display it persistently in the UI** next to the power indicator — an operator moving between two units must be able to see which limit is in force without opening a config file. Log it at startup and include it in any saved run record.

The original design read this from disk: the commented-out `ReadMiniXSetup()` in the reference parsed **Wattage Max** and **Safety Margin** from `MiniXCtrl.ini`, alongside **High Voltage kV Max** and the conversion factors. Someone later replaced the file-driven path with two hardcoded functions. Restoring a configuration file is a return to the original intent, not a novel design.

6.2 DAC write (AD5623R)

Command bytes — 2 unused bits, 3-bit command, 3-bit address (XXCCCAAA):

Channel	Byte	Function
A (HV)	0x18	Write and update DAC A
B (current)	0x19	Write and update DAC B

Both confirmed as write-**and-update**: commanded values take effect immediately.

```

counts = int(setpoint / factor / VRef * 4096)    # truncates toward zero
bhi     = (counts & 0xFF0) >> 4
blo     = (counts & 0x00F) << 4                  # 12 bits left-justified

```

Byte stream:

```

0x80, state & ~DACS | CLKSTATE, 0x7B    # assert DACS, CLK high
0x10, 0x02, 0x00, <0x18|0x19>, bhi, blo # clock out 3 bytes
0x80, state | DACS, 0x7B                 # deassert DACS

```

Note the state byte sets **CLKSTATE high** before asserting DACS — the reverse of the ADC path. The original has the opposite operation commented out directly above, indicating this was determined experimentally.

6.3 ADC read (MAX186 family) — completion

Control nibble, clocked out 4 bits MSB-first as **Start**, SGL/DIF, ODD/SIGN, MSBF:

Constant	Byte	Nibble clocked	Channel	Monitors
AD0	0xD0	1101	0	High voltage
AD1	0xF0	1111	1	Current

Only the upper 4 bits are clocked (0x13 with length 0x03); the low nibble is padding and never reaches the part. **Start**, SGL/DIF and MSBF are identical in both words — **ODD/SIGN alone selects the channel**. That single-bit difference explains the failure signature in §10: a mis-clocked nibble does not produce noise, it produces the *other channel's* value.

Byte stream (identical for both channels apart from the nibble):

```
0x86, 0x03, 0x00          # clock divisor - re-sent on every read
0x80, state & ~ADCS & ~CLKSTATE, 0x7B # assert ADCS, CLK LOW
0x13, 0x03, <AD0|AD1>     # 4 bits out, MSB first
0x80, state, 0x7B         # "INPUTMODE" - no-op in rev C0
0x20, 0x01, 0x00         # clock IN 2 bytes
0x80, state | ADCS, 0x7B  # deassert ADCS
```

Then read exactly 2 bytes back.

Four points about this sequence:

CLKSTATE is cleared before asserting ADCS — the reverse of the DAC path (§6.2), which sets it. The source comment reads “take ADC clock enable low – this makes clock high at the A/D,” implying the clock is inverted between the FT2232 and the converter. Do not normalize the two paths to a common idle level; each was determined against hardware.

The 0x80 ... INPUTMODE write between nibble and read is vestigial. In revision C0 INPUTMODE == OUTPUTMODE == 0x7B, so it only rewrites an unchanged state byte. On A0/A1/B0 it genuinely turned D1 around. Retain it: it costs one MPSSE command, and dropping it forecloses support for older boards.

No delay is inserted anywhere. The entire transaction — divisor, chip select, nibble, read, deselect — is a single **Write()**. MPSSE executes it back-to-back and the two result bytes accumulate in the RX buffer for a subsequent read. There is no host round-trip mid-transaction, and inserting one is not required.

SetTimeouts(40, 40) is re-applied immediately before each Read(), redundantly with initialization.

Unpacking

The reply is 1 null bit + 12 data bits + 3 trailing bits:

```
raw = ((rx[0] << 8) | rx[1]) >> 3      # 13 bits survive the shift
counts = raw & 0x0FFF                  # data
framing_ok = not (raw & 0x1000)         # null bit must be 0
volts = counts / 4096 * 4.096
```

The reference implementation computes the identical shift but **keeps all 13 bits unmasked** and divides directly by `DAC_ADC_Scale`. If the null bit ever came back set, the original would silently report a value above 4.096 V rather than flagging a framing error. The mask and the `framing_ok` test are a deliberate hardening, not a transcription difference — keep both.

Engineering units follow from §6.1: `kV = volts * HighVoltageConversionFactor`,
`µA = volts * CurrentConversionFactor`.

6.4 Monitor conversion and display

The reference monitor loop (`OnTimer`, event 1) does more than convert. Reproduce the parts that affect behavior, not the parts that only affect a Win32 dialog.

Per cycle, for each channel: read ADC → format `("%.4fV"` → `atof` → multiply by the conversion factor → add `1e-9` → format `("%.0fkV" / "%.0fuA"` → `atoi` to an integer. Separately, the unrounded double feeds a running average displayed to one decimal.

The `+1e-9` before `("%.0f"` is a deliberate nudge past the banker's-rounding boundary in MFC's formatter, matching the epsilon trick on the minima in §6.1. The round-trip through strings is an artifact of dialog-driven design; a Python port should carry doubles and format only at the presentation layer. **But note the consequence:** wattage is computed from the *rounded integers*, not the doubles —

```
maWattage = iVoltsIn * iMicroAmpsIn    # both already rounded to integer
```

— so the displayed power is quantized to $1 \text{ kV} \times 1 \text{ µA}$ products. If you compute wattage from the raw doubles instead, your numbers will disagree with the reference application by a few tens of mW. That is an improvement, but it is a *change*, and it matters because the same quantity drives the color banding below.

Averaging uses `AveMonitors(newValue, history[], isStart)` over a fixed window (the source uses a small odd window — 7 in the copy I was given; confirm the `NumAVE` constant in yours, as I did not see the `#define` directly). `isFirstMonitorAve` is set true whenever a new setpoint is commanded, which primes the window with the first sample instead of letting stale pre-change

readings drag the average. Reproduce that reset — without it, the display crawls toward a new setpoint over several seconds and looks like slow hardware.

Two independent presentations exist per channel: the integer (used for wattage and range logic) and the 1-decimal average (used for the monitor text). Keep them separate.

Wattage indicator banding

```
mwSafe      = 10
mwCaution  = (WattageMax - SafetyMargin * 2.0) * 1000
mwDanger    = WattageMax * 1000
```

Range	Color	4 W unit (mW)	10 W unit (mW)
< mwSafe	white	< 10	< 10
mwSafe – mwCaution	green	10 – 3900	10 – 9900
mwCaution – mwDanger	yellow	3900 – 4000	9900 – 10000
≥ mwDanger	red	≥ 4000	≥ 10000
otherwise	light blue	unreachable	unreachable

`mwSafe = 10` is an absolute floor and does **not** scale with the rating — on a 10 W unit the white band therefore covers only the bottom 0.1 % of range and is nearly unreachable in practice. Leave it at 10 unless you have reason to change it, but be aware the indicator behaves differently between variants at low power.

Note that `mwCaution` subtracts **twice** the safety margin while the setpoint clamp (§9.1) subtracts one. The yellow band therefore begins 50 mW below the highest commandable power on either unit, so a maximal setpoint displays yellow immediately. This is intended. On a 10 W unit that gap is 0.5 % of range rather than 1.25 %, making the yellow band proportionally narrower and easier to overshoot in one step.

The final branch is unreachable by construction; the source comments it “this should never happen, indicates error.”

Range checking

Each channel is tested against its expected setpoint (`dblExpectedHV`, `dblExpecteduA`, assigned *after* clamping) with a tolerance band of the form:

```
lower = expected * (1 - err) - 1.0
upper = expected * (1 + err) + 1.0
```

with `err` selected from a coarse and a fine threshold (10 % / 5 % in the copy supplied). The additive ± 1.0 is what makes low setpoints workable: at 5 μA a 5 % band is $\pm 0.25 \mu\text{A}$, which is below the 0.05 $\mu\text{A}/\text{count}$ resolution once

offset is included. **This is the mechanism that keeps §10's +4 % current readback from tripping a fault.**

A fault does not act immediately. A delay counter (`ErrTestDelay`, 7 cycles in the supplied copy) must expire with the channel continuously out of band before HV is dropped. Single-cycle excursions during ramp are ignored by design. Any reimplementation must keep this debounce, or it will trip on every setpoint change.

7. GPIO, HV enable, and interlock

7.1 Reading pins

0x81 -> 1 byte, ADBUS

0x83 -> 1 byte, ACBUS

Output bits read back their driven state, so 0x81 is a valid confirmation that the enable bits actually cleared — the teardown path in the test script relies on exactly this.

Observed values, serial 01300036:

Condition	ADBUS	ACBUS
HV off, idle	0001 1111	0000 0101
HV on, 15 kV / 10 μ A	1111 1111	0000 0101

ADBUS 0x1F idle = CLK, DataOUT, DataIN, ADCS, DACS all high (both chip selects correctly deasserted). Enabling sets 0x20, 0x40 and 0x80.

7.2 HV enable

Both `CTRL_HV_EN_A` (0x20) and `CTRL_HV_EN_B` (0x40) are set together to energize, cleared together to de-energize. Two bits for one function is a deliberate redundancy; treat writing only one as a defect, not a feature. The enable is a single 0x80 state write — the DACs hold their setpoints independently.

MONX (0x80, ADBUS7) is a live status output, not a pull-up. It reads 0 with HV off and 1 once the supply is up. It is therefore usable as a “tube ready” confirmation, and its failure to assert within a second or so of enabling is a genuine fault condition worth surfacing. This was not obvious from the source, where the bit is named `MON MINIX RDY` but never gated on.

7.3 Interlock

ACBUS0 (!RESET, 0x01): **1 = closed (safe to operate), 0 = open.** Despite the !RESET name the polarity is not inverted in the sense the name suggests —

verify against your own hardware before trusting a port, since an inverted read here fails dangerous.

Reference state machine, worth reproducing in full:

- **On transition to open:** disable all control buttons, dismiss any pending confirmation dialog by synthesizing an IDNO click, force HV off, disable again. The dialog-dismissal step exists so an operator cannot answer “yes” to an energize prompt that was raised before the interlock opened. In a Python GUI the equivalent is invalidating any in-flight confirmation, not merely graying the button.
- **On transition to closed:** HV is *not* re-enabled. A counter `indInterlockClear = 3` monitor cycles must elapse, showing an “interlock restored” indicator, before controls re-enable — and then only if HV is already off.

The interlock is polled in the monitor loop, so its response time is one monitor period. It is not an asynchronous cutout in software; the hardware presumably enforces its own. Do not represent it to users as instantaneous.

ACBUS2 (0x04) reads set in every observation and remains unidentified. ACBUS3 is TSCS, **active high** — the sole output in the high byte.

8. Temperature sensor (DS1722)

This is the least-verified path in this document. Setup and conversion are transcribed from source; the register-read transaction is **not** — see §12.

8.1 Setup

`SetTempSensor()` sets its own clock divisor inline, then:

```
0x10, 0x01, 0x00, TSCONFIG, TSCMD + 0x08    # 2 bytes out
0x82, high & ~TSCS, 0x08                    # TSCS taken LOW to close the write
```

`TSCMD + 0x08` selects continuous conversion, 12-bit resolution, no shutdown. Note the sequence *ends* by deasserting TSCS — and since TSCS is active high, “low” is the idle state. The chip select must be raised before the transfer; the retrieved excerpt shows only the trailing deassert, so confirm the leading assert in your copy.

8.2 Conversion

```
raw = (msb << 4) + (lsb >> 4)                # 12-bit, left-justified in 2 bytes
if msb >= 0x80:
    raw -= 4096                                # sign extend
temp_c = raw * 0.0625
temp_f = temp_c * 9.0 / 5.0 + 32.0
```

0.0625 °C = 1/16 is the standard DS1722 12-bit LSB, which corroborates the part identification. `TemperatureC` is cached as a member and surfaced through the status API.

Temperature is advisory in the reference application — nothing interlocks on it and no threshold appears in the source. If your GUI is intended for unattended operation, adding a thermal cutout is a reasonable enhancement, but note you would be inventing the threshold, not recovering it. The tube's own specification, not this document, should set it.

9. Setpoint sequencing and safety logic

9.1 Clamping and the wattage limit

Order of operations when a setpoint is committed:

1. Read both fields, `atof`.
2. Clamp each to `[min, max]`, setting `isOutOfRange` on any adjustment.
3. Compare each against its previous value to derive `isVoltageChanged` / `isCurrentChanged`.
4. If either changed and `iVolts * iMicroAmps >= SafeWattageMW`, reduce **current**: `iMicroAmps = SafeWattageMW / iVolts + 1e-9`. `SafeWattageMW` is derived from the configured rating (§6.1.1), so this clamp is the single point at which a mis-set rating becomes an over-power condition. Two defenses are worth adding beyond what the reference does:
5. **Bound-check the rating itself at startup**, not only the setpoint at commit time. A rating outside {4.0, 10.0} is a configuration error and should prevent the device from opening.
6. **Re-derive `SafeWattageMW` from `WattageMax` at every commit** rather than caching it. A cached value that outlives a configuration reload is a silent hazard.

Note also that the clamp compares the *product of clamped setpoints* against the limit, so on a 10 W unit the full 50 kV × 200 µA corner is reachable within 50 mW. The 4 W unit never approaches its DAC full scale, which means **the 10 W unit is the first configuration in which DAC saturation and the power limit coincide**. Verify the top of range deliberately if you intend to operate there.

5. If `isOutOfRange`, write the corrected values back into the input fields and `Sleep(200)` so the operator sees what actually got committed.
6. Assign `dblExpectedHV / dblExpecteduA` from the *clamped* values.

Step 4 always sacrifices current, never voltage — in all three branches, including the branch where only current changed. The voltage-reduction alternative is

present but commented out and dated 20080507 with the note “only change current for wattage correction.” This is a design decision about beam quality: the operator’s chosen kV is preserved and flux is given up. Preserve it, and make the adjustment visible rather than silent.

Note the comparison is $\geq$ against `SafeWattageMW` (3950 mW), so exactly 3950 mW triggers reduction. Combined with the epsilon in step 4, the committed product lands marginally below the limit rather than on it.

9.2 Ordering rules

Raising voltage: set current first, then voltage. Lowering: the reverse.

The intent is that the tube never transiently sits at both a high voltage and a high current. A port that writes both DACs in a fixed order regardless of direction will, on some transitions, briefly exceed the power limit. Determine the direction from the previous setpoint and order the two DAC writes accordingly, with a settling delay between them (200 ms in the reference; longer on NSI hardware via `dwSleepDelay / idxNSI`, whose exact values I did not recover — see §12).

Energizing: DACs to the commanded values → verify interlock closed → set both enable bits. Never enable first and then ramp the DACs, which would apply whatever the DACs last held.

De-energizing: DACs to zero (HV first, 200 ms, then current, 100 ms) → clear both enable bits → read back ADBUS and confirm `0x20` and `0x40` are clear. If the readback still shows them set, that is a hardware-level failure and the correct response is to tell the operator to check the tube physically, not to retry silently.

Startup: enumerate → open → MPSSE init (which clears the enables before anything else) → temp sensor → clock divisor → HV DAC = 0 → 200 ms → current DAC = 0 → 2 s settle.

The reference disables the monitor timer (`KillTimer`) for the duration of a setpoint change and restarts it afterward, which is also how it avoids the range checker firing mid-transition.

10. Empirical validation

10.1 Clock edge determination

An exhaustive sweep of the two plausible nibble-out opcodes against the two plausible byte-in opcodes, at a known setpoint of 15.0 kV / 10.0 μ A (1200 and 200 counts commanded):

Nibble	Read	ch0 / HV	ch1 / I	Verdict
0x13	0x20	1204	208	correct
0x13	0x24	1187	192	low ~1–1.5 %
0x12	0x20	184	209	ch0 collapsed
0x12	0x24	216	188	ch0 collapsed

0x13 (bits out) + 0x20 (bytes in) is correct, confirming the reference source’s choice. Framing bit clear in all four cases; the 13-bit unpack is validated (`0x25A2 >> 3 = 1204`, `0x0680 >> 3 = 208`).

The two failure modes are diagnostically distinct and worth remembering:

- Wrong **read** edge (0x24) samples data slightly stale and loses the low bits — a small negative bias, easy to mistake for calibration error.
- Wrong **nibble** edge (0x12) corrupts ODD/SIGN, so the channel select never lands and ch0 returns approximately ch1’s value. Both channels read “plausible,” which is why a naive nonzero test cannot distinguish it. **Only a cross-check against a known setpoint discriminates.**

10.2 Monitors do not zero with HV off

HV-off baseline returned 8–23 counts (0.008–0.023 V) across all four combinations — ADC offset and dither, not signal. Consequently:

- A test keyed on exact zero (`counts == 0`) will report a dead read path as working. The sweep script’s Stage A verdict text does this and its conclusion for the HV-off case should be disregarded; Stage B is what actually discriminated.
- A GUI must not treat a few counts on an unenergized tube as a fault, and must not treat nonzero monitors as proof that HV is on. **Use MONX and the enable-bit readback for that, not the monitors.**

10.3 Readback accuracy

At 15 kV / 10 μ A: HV read 1204 vs 1200 commanded (+0.33 %); current read 208 vs 200 (+4 %). The current channel at 10 μ A occupies only 200 of 4000 counts, so 8 counts is offset-and-quantization territory rather than a real error. This is precisely why the range bands in §6.4 carry an additive ± 1.0 term on top of the percentage — **do not tighten the tolerance to a pure percentage.**

10.4 Other confirmations

- MPSSE sync check: `0xAB` $\rightarrow$ `0xFA 0xAB`.
- Both DAC channels write-and-update; commanded values take effect without a separate update command. `--dac-opcode 0x11` was available as an alternative but `0x10` was used throughout and worked.

- Interlock read **CLOSED** throughout; the open path was **not** exercised on hardware (§12).
- Teardown enable-bit readback returned clear on every run.

10.5 Rating provenance

The 10 W rating of serial 01300036 was established from the unit’s hardware documentation, **not** from the device or the reference software, neither of which reports it. All hardware measurements in §10 were taken at 15 kV / 10 μ A — 150 mW, or 1.5 % of the confirmed 10 W rating — so nothing in this document validates behavior near either variant’s power limit. The clamp arithmetic in §9.1 and the color banding in §6.4 are unexercised above 150 mW.

11. Transaction serialization

Every peripheral shares one clock and one data line, with chip selects managed entirely by the host and no hardware arbitration. **Two overlapping transactions will corrupt each other**, and because the DAC path is one of them, corruption is not merely a bad reading.

The reference enforces this with a spin-lock around every write:

```
while(port_busy);
port_busy = 1;
status = Write(tx, pos, &ret_bytes);
port_busy = 0;
```

This is not thread-safe — it is a non-atomic test-and-set, and it does not cover the subsequent `Read()`, so a concurrent transaction could interleave between a write and its reply. It works in practice only because the reference is single-threaded with a message-pump timer.

A Python GUI is *not* in that position. A monitor loop on a timer plus operator-initiated setpoint writes plus a temperature poll are three independent sources of traffic. Requirements:

1. **One lock, held across the entire write-then-read pair**, not just the write. Use `threading.Lock` (or route all device access through a single worker thread with a command queue — preferable, since it also serializes naturally with the read).
2. **Purge before a transaction whose reply you intend to parse**. The reference does this at several points; stale bytes in the RX buffer will misalign every subsequent read.
3. **Do device I/O off the UI thread**. A 40 ms timeout plus a 4 ms latency timer per transaction, times two channels per monitor cycle, is enough to make a single-threaded GUI feel unresponsive.

4. **The safety shutdown path must be able to acquire the lock.** A blocked or deadlocked monitor thread must not prevent HV from being dropped. Bound every acquisition with a timeout and treat expiry as a reason to force the enables clear.

12. Known gaps

Items not established, listed so they are not mistaken for settled:

Item	Status
DS1722 register-read transaction	Setup and math transcribed; the read sequence supplying MSB/LSB to <code>GetTemp()</code> was not recovered. Untested end-to-end.
TSCS leading assert	Only the trailing deassert was seen in the excerpt. Confirm against source.
NumAVE, ErrTestDelay, tolerance percentages	Values 7, 7, and 10 %/5 % taken from the supplied copy; the <code>#defines</code> were not seen directly.
dwSleepDelay / idxNSI settling values	NSI hardware uses longer delays; the table was not recovered. 200 ms is a floor, not a verified value.
ACBUS2 (0x04)	Reads set in every observation. Function unknown.
Interlock-open behavior	Never exercised on hardware. The state machine is transcribed, not validated. Test this deliberately before relying on it.
0x11 DAC opcode	Untested; 0x10 confirmed working.

40 kV board | All validation was on a 50 kV unit (sn 01300036). The 40 kV constants are from source only. |

4 W variant | Never exercised. The 4 W constants are transcribed from source but were not run against 4 W hardware. |

10 W constants | **Derived, not transcribed.** The reference source contains no 10 W values; the table in §6.1.1 substitutes `WattageMax = 10.00` into the reference's own expressions. Validated only in that the 10 W rating of sn 01300036 was confirmed from hardware documentation — the derived `SafeWattageMW`, `mwCaution` and `mwDanger` values have not been exercised at the top of range. | Safety-margin scaling | `SafetyMargin` is absolute (50 mW) in the reference, giving different proportional headroom on the two variants (§6.1.1). Whether the vendor intended it to scale is unknown. |

FT2232 EEPROM user area | Never read. May or may not contain a rating

(§2.1). |
_RELEASE_1uA build | Lowers **CurrentMin** to 0.999999. Not exercised. |
Thermal limits | No threshold exists in the reference. Any cutout you add is
your own engineering judgment. |